applied sciences

MDPI

Article

Improving the Accuracy and Effectiveness of Text Classification Based on the Integration of the Bert Model and a Recurrent Neural Network (RNN_Bert_Based)

Chanthol Eang and Seungjae Lee *

Department of Computer Science and Engineering, Intelligent Robot Research Institute, Sun Moon University, Asan 31460, Republic of Korea; ngchanthol1@gmail.com
* Correspondence: leeko@sunmoon.ac.kr; Tel.: +82-010-3277-7524

Abstract: This paper proposes a new robust model for text classification on the Stanford Sentiment Treebank v2 (SST-2) dataset in terms of model accuracy. We developed a Recurrent Neural Network Bert based (RNN_Bert_based) model designed to improve classification accuracy on the SST-2 dataset. This dataset consists of movie review sentences, each labeled with either positive or negative sentiment, making it a binary classification task. Recurrent Neural Networks (RNNs) are effective for text classification because they capture the sequential nature of language, which is crucial for understanding context and meaning. Bert excels in text classification by providing bidirectional context, generating contextual embeddings, and leveraging pre-training on large corpora. This allows Bert to capture nuanced meanings and relationships within the text effectively. Combining Bert with RNNs can be highly effective for text classification. Bert's bidirectional context and rich embeddings provide a deep understanding of the text, while RNNs capture sequential patterns and long-range dependencies. Together, they leverage the strengths of both architectures, leading to improved performance on complex classification tasks. Next, we also developed an integration of the Bert model and a K-Nearest Neighbor based (KNN_Bert_based) method as a comparative scheme for our proposed work. Based on the results of experimentation, our proposed model outperforms traditional text classification models as well as existing models in terms of accuracy.

Keywords: RNN_Bert_based; KNN_Bert_based; Bert; RNN; SST-2 dataset

Citation: Eang, C.; Lee, S. Improving the Accuracy and Effectiveness of Text Classification Based on the Integration of the Bert Model and a Recurrent Neural Network (RNN_Bert_Based). *Appl. Sci.* **2024**, *14*, 8388. https://doi.org/10.3390/app14188388

Academic Editor: Fabrizio Marozzo

Received: 19 August 2024
Revised: 8 September 2024
Accepted: 13 September 2024
Published: 18 September 2024

1. Introduction

The goal of text classification in this paper is to enhance user experience in applications such as recommendation systems, chatbots, and virtual assistants. Understanding customer sentiment through accurate text classification helps improve products and services, tailor marketing strategies, and address customer concerns proactively. Building strong text classification models is crucial for achieving accurate sentiment analysis, automating tasks, and driving better business decisions. Emotional analysis via text classification is a crucial tool for understanding the complex web of people's opinions and emotions regarding diverse products, services, or topics. One widely employed methodology involves categorizing text into predetermined sentiment classes, encompassing positive, negative, neutral, and occasionally very positive or very negative sentiments. However, the pursuit of fine-grained sentiment classification, which delves into discerning nuanced gradations across multiple sentiment levels, presents a significant challenge. This necessitates navigating a spectrum of emotional subtleties, where sentiments may range from mildly positive or negative to intensely so. Consequently, a more sophisticated and nuanced approach to text analysis is required [1,2]. In recent years, deep learning models have had a significant impact on Natural Language Processing (NLP). Among them, Bert (Bidirectional Encoder Representations from Transformers) stands out. Bert is a powerful language model based on the Transformer architecture, and it has achieved state-of-the-art performance in various

NLP tasks. Researchers have embarked on an extensive exploration of Bert's efficacy in sentiment analysis, with a particular focus on benchmark datasets such as the Stanford Sentiment Treebank (SST) [3–5]. Unlike binary sentiment classification tasks that dichotomize sentiments into positive and negative labels, fine-grained sentiment classification presents a more nuanced challenge by delineating sentiments across five distinct classes: very negative, negative, neutral, positive, and very positive. The objective is to not only discern whether a text expresses sentiment but also to precisely gauge the intensity or degree of that sentiment [6]. Several studies have shown that pre-trained Bert models, followed by fine-tuning, provide robust performance in sentiment analysis tasks. These models outperform other popular approaches without requiring complex architectures [3,4]. In addition, transfer learning can leverage pre-trained language models, which have proven effective in NLP, similar to its success in computer vision [1]. Bert-based models improve the accuracy and effectiveness of sentiment analysis, especially when it comes to fine-grained sentiment classification. Researchers continue to explore variations and optimizations, leading to further advances in the field [7–9].

Bert's model is regarded as one of the most sophisticated approaches to text representation. In contrast to traditional models, which consider only one direction, Bert captures the context of a word in a sentence by considering both the words preceding and following it [10]. This bidirectional nature enables Bert to generate more sophisticated and precise representations of text, which is essential for tasks such as text classification. A Bayesian network is a probabilistic graphical model that represents a set of variables and their conditional dependencies through a directed acyclic graph. In the context of text classification, Bayesian networks can be employed to model uncertainty and dependencies between disparate features extracted from textual data. This approach offers a structured means of integrating prior knowledge and addressing intricate relationships [11–13]. A significant advancement in text classification methodologies was achieved by effectively combining deep contextual understanding, focused attention, and efficient feature extraction. The research is significant in that it could result in the development of more accurate, robust, and interpretable text classification systems, which are essential for a variety of applications in NLP. This hybrid approach demonstrates the potential for combining different models to leverage their unique strengths, thereby improving performance in complex text analysis tasks [14–16]. The objective is to enhance the efficacy of event text classification and event assignment through the utilization of Graph Convolutional Networks (GCN) and multi-attention mechanisms. GCNs are designed to operate on data represented as graphs, where nodes represent entities (such as words, phrases, or events) and edges represent relationships between these entities. In the context of event text classification, GCNs can be employed to model the relationships between disparate entities mentioned in the text, such as the connections between different events or the relationships between events and specific entities (e.g., people, and locations). GCNs facilitate the capture of structural dependencies and relational information which are essential for comprehending the context and categorization of events in a more comprehensive manner [17–19].

The aforementioned research indicates that there are many methods for enhancing the precision of text classification models by integrating the Bert model with various deep learning models. However, in this work, we integrate the Bert model with the RNN model, which represents our primary proposed model in this paper. The combination of the two models results in an enhancement in text classification due to the complementary strengths of both models. Bert offers a profound, bidirectional comprehension of words, elucidating their meanings based on the context of surrounding text. RNN, on the other hand, demonstrates exceptional proficiency in processing sequential information and discerning the order and flow of words within a sentence. The integration of Bert with the RNN model provides the following benefits:

- Bidirectional Context: Bert offers a comprehensive, context-based understanding of word meanings.

- Sequential Processing: RNN is particularly adept at identifying the order and flow of words in a sequence.
- Enhanced Accuracy: Combining Bert's contextual embeddings with RNN's sequential modeling improves text classification accuracy.
- Improved Generalization: The combined model enhances the ability to generalize across different text types.

The remainder of this paper is structured as follows: Section 2 presents the related work concerning various text classification models. Section 3 presents the proposed methodology, which will cover Bert and the proposed RNN_Bert_based model structure. The proposed solution flowchart and algorithm are also included in Section 3. Section 4 presents the simulation settings, parameters, and the existing work on text classification. It also explains the experimentation of the proposed RNN_Bert_based algorithm, simulation setting, and its result. Finally, a conclusion of our work contributions and the significant task for future research are presented in Section 5.

2. Related Work

Based on [20], the authors introduced an innovative approach to sentiment analysis by leveraging the capabilities of Bert in conjunction with Bidirectional Long Short-Term Memory (BiLSTM) and Bidirectional Gated Recurrent Unit (BiGRU) algorithms. The core idea is to create hybrid deep learning architectures that seamlessly integrate the contextual understanding of Bert with the sequential modeling capabilities of BiLSTM and BiGRU. The main goal of the study is to improve the accuracy and efficiency of sentiment analysis tasks by leveraging the complementary strengths of these different neural network architectures. Bert, known for its contextualized word embeddings and bidirectional encoding, excels at capturing intricate linguistic nuances and understanding the context of a given text. On the other hand, BiLSTM and BiGRU models are adept at capturing sequential and long-range dependencies within the input data, making them well-suited for tasks involving sequential data processing. This research provides a significant advancement in sentiment analysis methodologies, demonstrating the potential of hybrid deep learning architectures to improve the accuracy and robustness of sentiment classification systems. By combining the strengths of Bert with the BiLSTM and BiGRU algorithms, the proposed models offer a promising avenue for further advances in natural language understanding and text analysis applications.

In [21], the author proposed a novel approach to sentiment analysis using Bert, a powerful language representation model. It presents an ensemble method and a compressed Bert model, designed to improve the accuracy of sentiment analysis. These approaches outperform existing tools by 6–12% on the F1 score measure across three datasets. The study highlights the importance of integrating advanced NLP techniques such as Bert into sentiment analysis pipelines, offering the potential for more accurate and efficient sentiment analysis tools from a software engineering perspective.

In [22], the authors investigated the effectiveness of Bert in sentiment analysis tasks after fine-tuning. The primary objective is to evaluate Bert's performance in understanding and classifying sentiment from text data, especially after adapting its pre-trained representations to the specific nuances of sentiment analysis. It examines Bert's ability to understand sentiment nuances in text data by adapting its pre-trained representations to specific sentiment analysis contexts. The research demonstrates Bert's robust performance after fine-tuning and highlights its superior accuracy in classifying sentiment across different classes. Overall, the study underscores the transformative potential of deep learning-based models like Bert in advancing sentiment analysis capabilities and understanding human emotions expressed in textual data.

Next, according to [23], the authors introduced a novel approach to improve sentiment analysis models by enhancing word embeddings with sentiment information. The main idea revolves around the notion that incorporating sentiment-related features into word embeddings can enrich the semantic understanding of words and consequently improve

the performance of sentiment classification models. The core concept of the study is to augment traditional word embeddings with sentiment-specific information, thereby providing a deeper understanding of the emotional connotations of words. This sentiment-enhanced word embedding method involves associating sentiment scores or labels with individual words during the embedding process, allowing the model to learn representations that capture both semantic and sentiment-related aspects of words. By integrating sentiment information directly into word embeddings, the proposed method seeks to address the inherent limitations of traditional word embeddings in effectively capturing sentiment nuances. Traditional word embeddings, such as Word2Vec or GloVe, often struggle to adequately represent sentiment-related variations in word meanings due to their purely semantic nature. Through empirical evaluation and comparative analysis, the study demonstrates the effectiveness of the sentiment-enhanced word embedding method in improving sentiment analysis models. By training sentiment classification models with these enriched embeddings, the research demonstrates improvements in classification accuracy and performance on various sentiment analysis tasks and datasets.

As indicated in [24–26], the three papers demonstrate the application of sophisticated NLP methodologies to effectively address significant challenges in text classification and sentiment analysis. In [24], the authors improved the classification of lengthy Chinese news articles by combining Bert's deep contextual understanding with CNN's capacity to extract crucial local features, resulting in a more precise categorization of extensive texts. Next, according to [25], the authors addressed the significant issue of data imbalance in text classification by constructing contrastive samples, which result in models that are not only more accurate but also more equitable in their predictions, while in [26], the authors represented a significant advancement in the field of sentiment analysis, particularly in the context of Douban film comments. It employs a sophisticated hybrid model that combines Bert, CNN, BiLSTM, and an attention mechanism, thereby enhancing both the accuracy of sentiment detection and the interpretability of the results. Collectively, these studies contribute significantly to the ongoing development and refinement of NLP techniques, offering robust solutions to complex problems in the field. In [27–30], readers will find an exploration of cutting-edge approaches to improving text classification and sentiment analysis across a variety of challenges. This work concentrates on the process of fine-tuning Bert for the purpose of multi-label sentiment analysis in the context of unbalanced, code-switching texts, with a view to addressing the inherent complexities associated with multilingual data. The second journal presents an interactive multitask learning approach to sentiment classification in Chinese text, which enhances model performance by leveraging related tasks. The third journal presents a method for learning label-adaptive representations, which is crucial for large-scale multi-label text classification. This method improves the model's ability to handle vast and diverse label sets. The fourth journal addresses the detection of machine-generated text through adversarial fine-tuning of pre-trained language models, a crucial aspect in differentiating between human-written content and AI-generated text. Collectively, these journals make significant contributions to advancements in sentiment analysis, multi-label classification, and the detection of synthetic texts, addressing some of the most pressing challenges in NLP.

In [31–33], a significant contribution was made to the advancement of text classification and sentiment analysis, addressing diverse challenges through innovative methodologies. The initial journal presents a three-branch Bert-based text classification network, specifically designed for the analysis of gastroscopy diagnosis texts, which enhances the accuracy of medical text interpretation. The second journal investigates the use of fine-tuning transformer models for multilingual identification of threatening texts, employing transfer learning to enhance detection across different languages and improve security measures. The third journal introduces a transformer-based efficient model for transfer learning and language understanding, which optimizes performance in various NLP tasks by improving the efficiency and adaptability of transformer models. This journal also develops an automatic sentiment analysis method for short texts using a hybrid Transformer-Bert model,

enhancing sentiment detection in brief and often contextually sparse text. Collectively, these journals represent significant advancements in the application of advanced NLP techniques to specialized domains, multilingual applications, and efficient text analysis, offering robust solutions to contemporary challenges in text processing and understanding. In [34,35], the field of text classification is significantly advanced by using state-of-the-art methods to address complex challenges. The first paper explores the use of language models, including Bert and GPT-4, combined with attention mechanisms for the hierarchical classification of radiology reports. This approach improves the accuracy of medical text classification by effectively managing the complex structure and specialized terminology of radiology reports, leading to improved diagnostic support. The second paper focuses on an improved convolutional neural network (CNN) algorithm for text classification, which optimizes CNN performance to better capture relevant features and patterns in text data. Together, these studies represent important developments in the application of sophisticated language models and advanced neural network techniques to improve classification tasks in specialized medical contexts and general text processing, thereby increasing both accuracy and efficiency in handling complex textual information. In [36], the authors proposed a novel machine learning technique called one-class learning, which is employed to differentiate between AI-generated essays and human-written ones. This research presents a practical solution to the problem of distinguishing between AI-generated and human-written essays by leveraging one-class learning, which requires less labeled data and can be effective in identifying deviations from a known norm. Finally, in [37], the authors proposed a deep one-class (DOC) classification method, identifying whether a test sample belongs to a particular class or not, given only the data of that single class. This work introduces a deep-learning approach for one-class transfer learning, leveraging labeled data from unrelated tasks to enhance feature learning in one-class classification. The method builds on a convolutional neural network (CNN) to produce features that are both descriptive and exhibit low intra-class variance for the target class. Extensive experiments across various datasets demonstrate that this deep one-class (DOC) classification method significantly outperforms state-of-the-art techniques in anomaly detection, novelty detection, and mobile active authentication. The above methods are all text classification models based on traditional classification models and deep learning models, including the integration of Bert with deep learning architectures such as CNN. However, in our work, we integrate the RNN with the Bert model, which allows us to achieve better performance compared to existing methods and traditional approaches. This improvement is due to the RNN's ability to capture sequential dependencies and contextual relationships within the text, which is particularly beneficial when combined with Bert's pre-trained contextual embeddings. While CNNs excel at capturing local features, RNNs are more adept at modeling the temporal structure of sequences, making them better suited for tasks where understanding the order of words or phrases is critical. This combination allows for a more nuanced and accurate representation of text, resulting in superior classification performance.

3. Proposed Methodology

In order to successfully implement the RNN_Bert_based algorithm, it is necessary to utilize the traditional Bert model, which will be integrated with a Recurrent Neural Network (RNN). The objective of this integration is to enhance the model's capacity to identify and capture sequential dependencies within text data. The SST-2 dataset, which is renowned for its binary sentiment classification tasks, will serve as the principal benchmark for evaluating the algorithm's performance. The integration of Bert contextual embeddings with RNN sequential learning capabilities is expected to enhance the model's accuracy and robustness in sentiment analysis tasks. In this paper, we developed an algorithm that outperforms the existing work with the method of "Convolutional Neural Networks for Efficient Text Classification and Robus Bidirectional Encoder Representations from Transformers (TextCNN + RoBERTa)" as described in the paper reference [14]. The experimental

results show that our proposed RNN_Bert_based method achieves higher performance than the existing work in terms of model accuracy, precision, F1 score, and recall.

3.1. Dataset Structure

The SST-2 dataset is a popular benchmark dataset widely used in NLP tasks, particularly for sentiment analysis. This dataset appears to represent a syntactic parse tree, or constituency tree, commonly used in NLP to represent the grammatical structure of sentences. Each number corresponds to a particular syntactic category or level of nesting within the sentence, and the brackets indicate hierarchical relationships between phrases or words. For example, extracting from the first part of the validation set, it describes a sentence like "It's a nice movie with nice performances by Buy and Accorsi". The nested numbers probably represent phrase structures, where 2 may represent a noun or verb phrase, 3 could be an adjective phrase, and 4 could represent deeper nested structures within the sentence. This type of data structure is often seen in treebanks, such as the Penn Treebank, where each sentence is parsed into a tree structure that reflects its syntactic composition. It is useful for training models for syntactic parsing or for tasks that rely on understanding sentence structure, such as semantic analysis or sentiment classification. Each phrase and word are tagged with a sentiment score ranging from 1 to 5, as follows: 1: very negative; 2: negative; 3: neutral; 4: positive; and 5: very positive. For example, we extract from "(4 (3 (2 A) (4 poignant)) (3 (2 ,) (4 (4 artfully) (3 (2 (3 (3 crafted) (2 meditation)) (2 (2 on) (2 mortality))) (2 .)))))" in which the sentiment labels are placed at the beginning of the nodes, with the entire sentence receiving an overall sentiment of 4 (positive), 2 (negative), and 2 (neutral). Overall, this sentence shows a positive sentiment because the word "poignant" and the overall combination of phrases tilt the sentence towards a positive score of 4. The dataset is derived from movie reviews, with sentences extracted from movie reviews labeled as either positive or negative based on the sentiment expressed.

SST-2 consists of a comprehensive set of sentences extracted from movie reviews, reflecting the diverse expressions of sentiment within cinematic discourse. Each sentence is carefully paired with a sentiment label that indicates whether the sentiment conveyed is positive or negative. This meticulous annotation process ensures the reliability of the dataset and facilitates accurate evaluation of sentiment analysis models. By including a wide variety of sentences and sentiment labels, SST-2 serves as an invaluable resource for exploring the complexities of sentiment classification in natural language and advancing the capabilities of NLP systems. SST-2, constructed from an extensive pool of movie reviews, presents a rich tapestry of sentences that encapsulate diverse moviegoing experiences and opinions. These sentences encapsulate a spectrum of emotions, ranging from glowing endorsements to scathing critiques. Each sentence is meticulously paired with a sentiment label, carefully crafted to reflect the nuanced polarity of the sentiment expressed, whether it be an enthusiastic endorsement or a critical appraisal. This meticulous annotation process ensures the reliability of the dataset and facilitates accurate evaluation of sentiment analysis models. By including a wide variety of sentences and sentiment labels, SST-2 serves as an invaluable resource for exploring the complexities of sentiment classification in natural language and advancing the capabilities of NLP systems. Figure 1 is a sample of some of the sentences extracted from the dataset.

SST-2 uses a binary sentiment classification approach that categorizes sentences as positive or negative, simplifying sentiment analysis into unambiguous categories. This framework enhances usability and aligns with real-world applications by emphasizing the essential linguistic cues and semantic features underlying sentiment expression.

No	Text
1	(3 (2 It) (4 (4 (2 's) (4 (3 (2 a) (4 (3 lovely) (2 film))) (3 (2 with) (4 (3 (3 lovely) ➢ "It's a lovely film with lovely performances by Buy and Accorsi.
2	(2 (2 (1 No) (2 one)) (1 (1 (2 goes) (2 (1 (2 (2 unindicted) (2 here)) (2 ,)) (2 (2 which) (3 (2 (2 is) (2 probably)) (3 (2 for) (4 (2 the) (4 best))))))) (2 .))) ➢ "No one goes unindicted here, which is probably for the best."
3	(4 (3 (2 A) (4 poignant)) (3 (2 ,) (4 (4 artfully) (3 (2 (3 (3 crafted) (2 meditation)) (2 (2 on) (2 mortality))) (2 .))))) ➢ "A poignant, artfully crafted meditation on mortality."
4	(4 (4 (2 A) (4 (4 (2 fast) (4 (2 ,) (4 (3 funny) (4 (2 ,) (4 (2 highly) (3 enjoyable)))))) (2 movie))) (2 .)) ➢ "A fast, funny, highly enjoyable movie."
5	(2 (3 Makes) (3 (2 (2 even) (2 (2 the) (2 (1 claustrophobic) (2 (2 on-board) (2 quarters))))) (3 (3 (2 seem) (4 fun)) (2 .)))) ➢ "Makes even the claustrophobic on-board quarters seem fun."

Figure 1. Sample sentences from the SST-2 dataset for analysis and understanding.

3.2. Bert Model Structure

The text classification model is based on Bert (Bidirectional Encoder Representations from Transformers), and the robust Bert mechanism leverages state-of-the-art language representation techniques for improved performance. The architecture of this model, shown in Figure 2, consists of five keys:

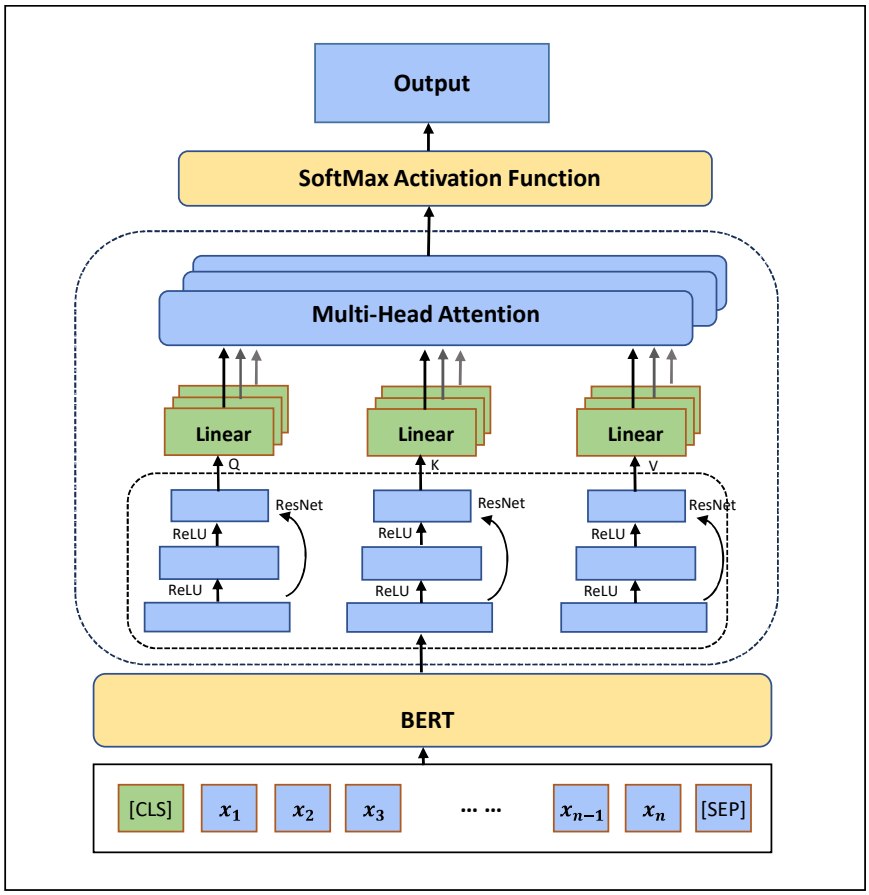


Figure 2. Architecture and components of Bert model structure for text classification.

3.2.1. Input Layer

The input layer of the text classification model based on Bert and the robust Bert mechanism play a crucial role in processing raw text data. Its responsibilities go beyond

simply receiving the text, and it performs several key tasks as follows to prepare the input for further processing:

- Text Preprocessing: The input layer begins by receiving the raw text data, which may include unstructured text from various sources such as documents, social media posts, or reviews. It performs initial preprocessing tasks such as removing punctuation, handling capitalization, and tokenizing the text into individual words or sub-word units.
- Tokenization: Once the text has been preprocessed, the input layer applies tokenization, which breaks the text into discrete units called tokens. These tokens typically correspond to words, subwords, or characters, depending on the tokenization strategy chosen. In the case of Bert, WordPiece Tokenization is commonly used, where the text is segmented into subword units based on a predefined vocabulary.
- Token Representation: After tokenization, each token is mapped to a unique numeric representation using an embedding lookup table. This process converts the textual tokens into dense vector representations, often called word embeddings or token embeddings. These embeddings capture semantic information about the tokens and facilitate the model's understanding of the input text.
- Sequence padding: To ensure uniform input dimensions, the input layer can apply sequence padding to the tokenized text. This involves adding padding tokens (usually zeros) to shorter sequences to ensure that all input sequences have the same length. Padding allows for efficient batch processing and matrix computation within the neural network architecture.

In essence, the input layer serves as the gateway for raw text data into the neural network, orchestrating essential preprocessing steps to transform the text into a format suitable for further processing by subsequent layers. Its meticulous handling of text data sets the stage for effective learning and feature extraction by the subsequent layers of the model.

3.2.2. Bert Encoding Layer

The Bert encoding layer, a crucial component of the text classification model, performs a complex transformation of the input text into comprehensive vector representations that encapsulate rich contextual information. This section will provide a more detailed examination of its functionalities:

- Contextual Embeddings: In contrast to traditional word embeddings, which assign fixed representations to individual words irrespective of context, the Bert encoding layer generates contextual embeddings. These embeddings capture the nuanced meanings of words within the context of the entire input sequence, thereby enabling the model to comprehend subtle semantic nuances and disambiguate polysemous words.
- Transformer Architecture: The Bert encoding layer employs the Transformer architecture, a self-attention mechanism that enables the capture of long-range dependencies and contextual relationships within the input text. Through the use of multi-head self-attention mechanisms and position-wise feedforward networks, Bert is capable of effectively processing input sequences, thereby capturing both local and global contextual information.
- Pre-trained Language Representations: Bert is typically pre-trained on large-scale text corpora using unsupervised learning objectives, such as masked language modeling and next sentence prediction. During pre-training, Bert learns to encode text into rich, contextualized representations by optimizing for various language understanding tasks. These pre-trained representations serve as a foundation for downstream tasks, including text classification, enabling the model to leverage extensive linguistic knowledge acquired during pre-training.
- Fine-tuning for Specific Tasks: While Bert's pre-trained representations capture general linguistic patterns, the encoding layer can be fine-tuned on task-specific data through supervised learning. This process involves updating the parameters of the Bert model

using labeled data from the target task, such as sentiment analysis. This adaptation of the model's representations enhances performance on specific classification tasks.

- High-dimensional Vector Representations: The Bert encoding layer outputs high-dimensional vector representations for each token in the input sequence. These representations typically have hundreds of dimensions, capturing a wealth of information about the semantics, syntax, and context of the input text. The richness of these representations enables downstream layers of the model to make informed decisions during classification tasks.

The Bert encoding layer serves as a robust and scalable foundation for contextualized text representations. It employs transformer-based architectures and pre-trained language models to transform input text into high-dimensional vector representations. Bert's capacity to capture rich contextual information enables Bert-based models to achieve state-of-the-art performance across a diverse range of NLP tasks, including text classification, sentiment analysis, and named entity recognition.

3.2.3. The Fully Connected Layers

The fully connected layers within the text classification model play a pivotal role in the extraction and transformation of the encoded text features generated by preceding layers, such as the Bert encoding layer. An in-depth examination of their functionalities is follows:

- Multi-scale Feature Extraction: The incorporation of multiple fully connected layers enables the model to extract text features at varying scales or levels of abstraction. Each fully connected layer can capture distinct aspects of the input text, ranging from fine-grained details to higher-level semantic information. This multi-scale feature extraction facilitates the model's ability to discern intricate patterns and semantic nuances present in the text across different levels of granularity.
- Dimensionality Transformation: Fully connected layers facilitate the transformation of feature vector representations from one dimension to another. This transformation serves two primary purposes. Dimensionality Expansion: In some cases, fully connected layers increase the dimensionality of the feature vector representation, allowing it to capture more complex patterns and relationships within the input text.
- Dimensionality Reduction: Conversely, fully connected layers can also reduce the dimensionality of the feature vector representation, condensing it into a lower-dimensional space while retaining essential information. Dimensionality reduction techniques, such as principal component analysis (PCA) or feature selection, are employed with the objective of preserving the most salient features of the input text while reducing computational complexity and mitigating the risk of overfitting.
- Nonlinear Transformation: Fully connected layers typically incorporate nonlinear activation functions, such as ReLU (Rectified Linear Unit) or sigmoid functions, to introduce nonlinearity into the feature transformation process. This nonlinearity enables the model to capture complex relationships and interactions between different features, enhancing its capacity to model intricate textual patterns and semantic structures.
- Feature Fusion and Abstraction: As the feature vectors propagate through the fully connected layers, they undergo successive transformations that fuse and abstract information from the input text. By iteratively combining and refining features across multiple layers, the model learns to extract hierarchical representations of the input text, gradually transitioning from raw textual data to high-level semantic abstractions.

3.2.4. Residual Networks

The incorporation of a residual network [38] (ResNet) layer within the text classification model represents a deliberate effort to enhance both the performance and stability of the model architecture. This section will examine the specific functions and advantages of this layer in greater detail. ResNet is a deep learning model that employs residual blocks to construct deep neural networks. Each residual block contains a skip connection, which enables the direct transfer of input data to the output, effectively mitigating the gradient

disappearance and gradient explosion issues commonly observed in deep neural networks. To prevent the downstream classification network from deteriorating and to enhance the model's stability, the residual structure is incorporated into the model to extract multiscale text segment features. The residual network formula is presented in Equation (1).

$$y = F(x) + x \tag{1}$$

$$F(x) = max(0,\ xW_1 + b_1)W_2 + b_2 \tag{2}$$

where x denotes the input data and $F(x)$ denotes the transform operation in the residual block. The transformation operation taken in this paper is shown in Equation (2). The output y can be obtained by adding x to the transformation operation $F(x)$. This process can be seen as a correction to the input x, thus making it easier for the model to learn the important information in the input x. The specific functions and advantages of this layer are as follows:

- Performance Enhancement: ResNet layers are renowned for their ability to facilitate the training of very deep neural networks. By incorporating residual connections, where the output of one layer is added to the input of another, ResNet layers alleviate the vanishing gradient problem encountered in deep networks. This enables more efficient gradient flow during training, facilitating the learning of intricate patterns and representations within the input text.
- Stability Improvement: The residual connections within the ResNet layer introduce skip connections that bypass certain layers in the network. This allows the model to retain information from earlier layers, mitigating the risk of information loss or degradation as the input propagates through successive layers. Consequently, ResNet layers promote greater stability and robustness in the model's predictions, reducing the likelihood of overfitting and improving generalization performance.
- Detailed Information Capture: By summing the outputs of different fully connected layers with the input vectors, the ResNet layer effectively integrates detailed information from multiple scales and abstraction levels. This enables the model to capture fine-grained textual features and semantic nuances present in the input text, enhancing its discriminative power and expressive capacity.
- Hierarchical Feature Fusion: The residual connections within the ResNet layer enable hierarchical feature fusion, allowing the model to combine information from different layers and abstraction levels. This facilitates the integration of both low-level and high-level features, enabling the model to capture complex relationships and interactions within the input text.

3.2.5. The Output Layer

The output layer, situated at the pinnacle of the model architecture, plays a pivotal role in transforming the voluminous feature representations acquired throughout the network into actionable predictions. Let us now examine the functionalities and mechanisms employed within the output layer in greater detail.

- The integration of the mechanism output is as follows: the output of the preceding layers, which frequently includes the contextual embeddings derived from Bert as well as the aggregated information from the residual network, serves as the input to the output layer.
- Fully Connected Transformation: The input from the preceding layers undergoes transformation through one or more fully connected layers within the output layer. These fully connected layers serve to further distill and refine the learned representations, enabling the model to extract high-level features and patterns relevant to the classification task at hand.
- Softmax Activation: Following the transformation through fully connected layers, the output of the final fully connected layer is passed through a softmax activation function. The softmax function computes the probability distribution over the possible

classes or categories, assigning a probability score to each class. This probability distribution reflects the model's confidence in each class, with higher probabilities indicating stronger predictions.

- Classification Output: The final step in the output layer involves the classification of the input text based on the computed probability distribution. The class with the highest probability score is selected as the predicted class label for the input text. This classification output serves as the model's prediction for the sentiment, category, or label associated with the input text, enabling downstream applications to utilize the model's insights for decision-making or analysis.
- Training and Optimization: During the training phase, the output layer is optimized to minimize the discrepancy between the predicted probability distribution and the ground truth labels. This is typically achieved through the use of a loss function, such as cross-entropy loss, which quantifies the difference between the predicted probabilities and the true labels.

The output layer serves as the final stage in the model's decision-making process, transforming the learned feature representations into probabilistic predictions. By integrating information from preceding layers, applying transformations through fully connected layers, and computing probability distributions via softmax activation, the output layer enables the model to make informed classifications and predictions based on the input text.

3.2.6. Word Embedding with Bert Layer

In the context of NLP, the quality of text feature representation is of paramount importance for the effectiveness of the task in question. In this paper, we employ word embedding to segment the input text into independent words and map them into unique corresponding tokens.

$$e(w_i) \epsilon \mathbb{R}^d \tag{3}$$

where w_i denotes the $i-th$ word, $e(w_i)$ denotes the embedding vector of the word, and $\mathbb{R}^d$ denotes the $d-$ dimensional—vector space.

The main feature of the Bert pre-trained language model is the use of the Transformer bidirectional encoder, whose structure is shown in Figure 3. The Bert model is designed to process input sentences composed of multiple words, with a maximum length of 512 tokens. Prior to input, each sentence undergoes preprocessing, which involves the addition of a [CLS] token at the beginning to indicate the sentence's commencement and the insertion of a [SEP] token at the end to signify its conclusion. This ensures the clear delineation of sentence boundaries and facilitates subsequent processing. Following tokenization and addition of special tokens, the input text is further processed to create three distinct types of input vector embeddings as follows, as depicted in Figure 4:

- Word Vectors: These embeddings represent individual tokens (words or subwords) within the input sentence. Each token is mapped to a corresponding vector representation, which is learned during pre-training. This representation encodes semantic information about the token's meaning and context.
- Segment Vectors: Bert employs segment vectors to process input sequences comprising multiple sentences or segments. These vectors assign a unique identifier to each token, indicating its segment or sentence of origin. This enables the model to distinguish between different segments and consider their contextual relevance during processing.
- Location Embedding Vectors: In addition to word and segment embeddings, Bert incorporates location embedding vectors to encode positional information within the input sequence.

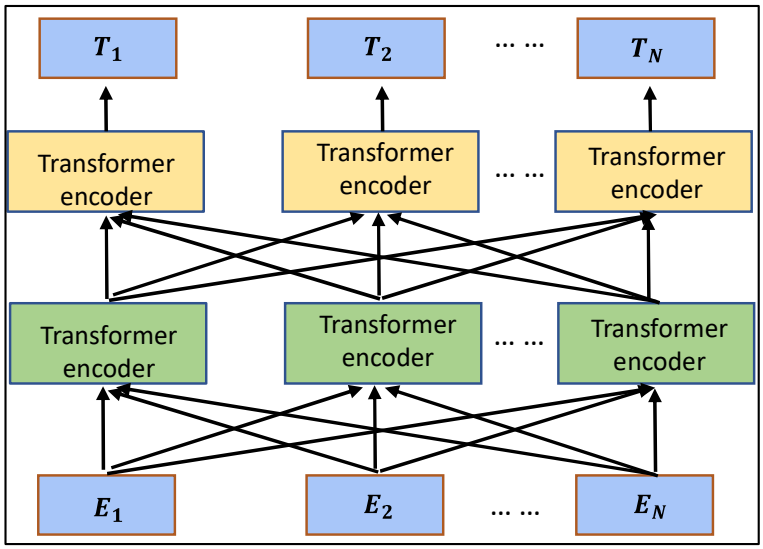


Figure 3. Bert model structure diagram.

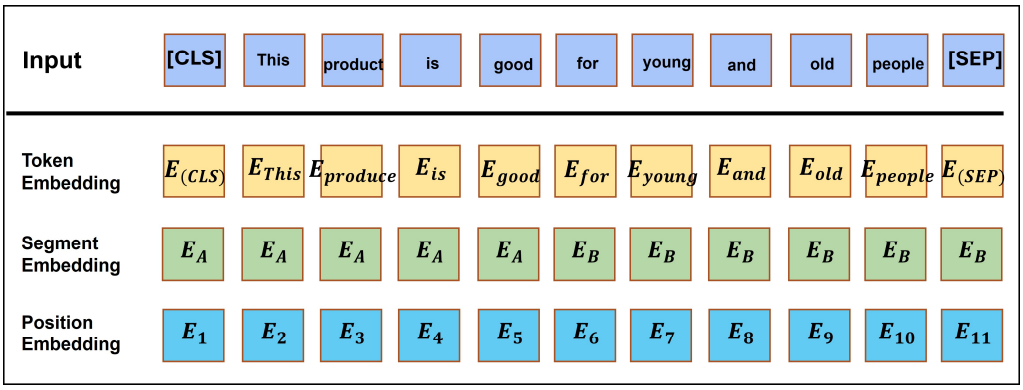


Figure 4. Bert model tokenization process.

3.3. Bert and K-Nearest Neighbor Based (Bert_KNN_Based) Model

Bert's embeddings capture the semantic meaning of the text, allowing KNN to work with high-quality feature representations. This leads to better performance compared to traditional text representations like bag-of-words. The embeddings provided by Bert reflect contextual similarities. As a result, the distance metrics used by KNN (e.g., Euclidean distance) become more meaningful and effective for finding semantically similar texts. KNN provides a clear rationale for its classifications by showing which neighbors influenced the decision. This interpretability is enhanced by the semantic richness of Bert embeddings, making it easier to understand why certain texts are classified in a particular way. However, distance computation is a drawback of this integration as KNN requires calculating distances or similarities between the query embedding and all stored embeddings. For large datasets, this can be computationally expensive and may result in slow query times. This can slightly affect the model's performance too. We can consider a small dataset with the K-Nearest Neighbors Bert based model (KNN_Bert_based) because the classification performance can be better than the traditional scheme. Figure 5 shows the integration of the Bert model with KNN. This integration leverages the powerful feature extraction capabilities of Bert, combined with the simplicity and effectiveness of the KNN algorithm.

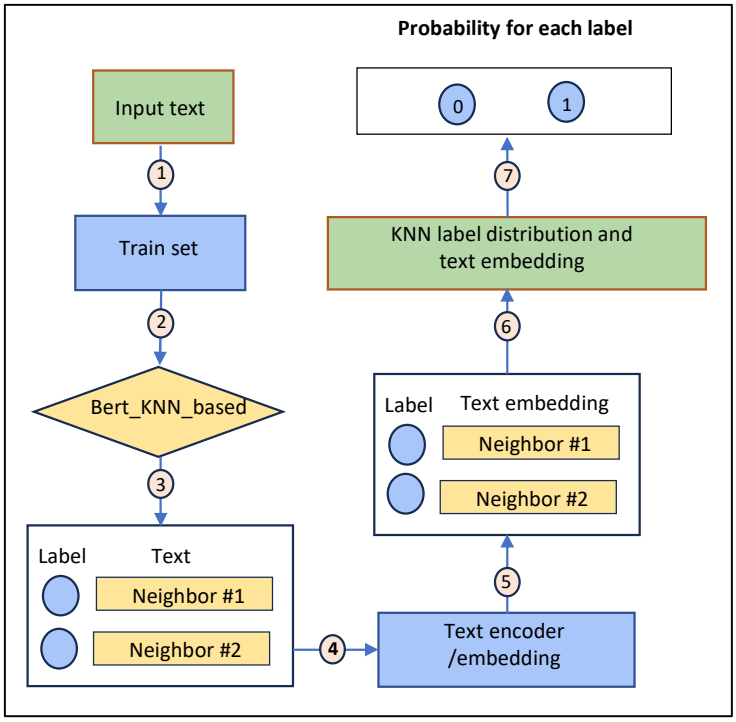


Figure 5. Integration of the Bert model with KNN.

3.4. Proposed Bert and Recurrent Neural Network Based (RNN_Bert_b)

The combination of Bert and RNN for text classification tasks, such as the SST-2 sentiment analysis dataset, effectively leverages the strengths of both models. Bert provides deep contextual embeddings that capture the nuances of words in their specific contexts, while RNN excels at capturing the dependencies and order within the sequence of embeddings. This combination typically yields enhanced performance on classification tasks, as the model benefits from Bert's robust contextual understanding and RNN's sequential learning capabilities. Furthermore, RNNs, especially when enhanced with attention mechanisms, can handle longer sequences more effectively than transformers alone, making this hybrid approach particularly effective for complex text classification challenges. Attention mechanisms have been introduced to enhance the performance of RNNs by enabling the model to focus on specific elements of the input sequence, rather than processing all the information equally. The attention mechanism functions by assigning attention scores to various elements of the input sequence. The attention scores are typically calculated as a weighted sum based on the relevance of each input token (word, for example) to the current output. RNN has difficulty processing long-term dependencies due to issues such as vanishing or exploding gradients. This makes it challenging for them to capture information from distant time steps. The attention mechanism provides a solution to this problem by enabling the model to focus on specific relevant time steps, regardless of their distance in the past. Figure 6 shows the flowchart of integration of Bert and RNN model.

The hyperparameters that control the behavior and performance of the RNN model combined with Bert are as follows: The maximum length of the input sequences (max_seq_len) has been set to 50. The number of training iterations over the entire dataset (epochs) is set to five. The number of samples processed in a single batch is set to 16 and 32. The learning rate (lr), which represents the step size at each iteration while moving toward a minimum of the loss function, is set to 2×10^{-5}, 3×10^{-5}, and 5×10^{-5}. The number of epochs without improvement in validation accuracy before stopping the training early (patience) is set to 1, and the maximum norm of the gradients for clipping (max_grad_norm) is set to 10. Next, the number of features in the hidden state of the RNN model, which determines the capacity of the RNN to learn temporal dependencies in the data is set to 256, and the number of classes in the classification task is set to two.

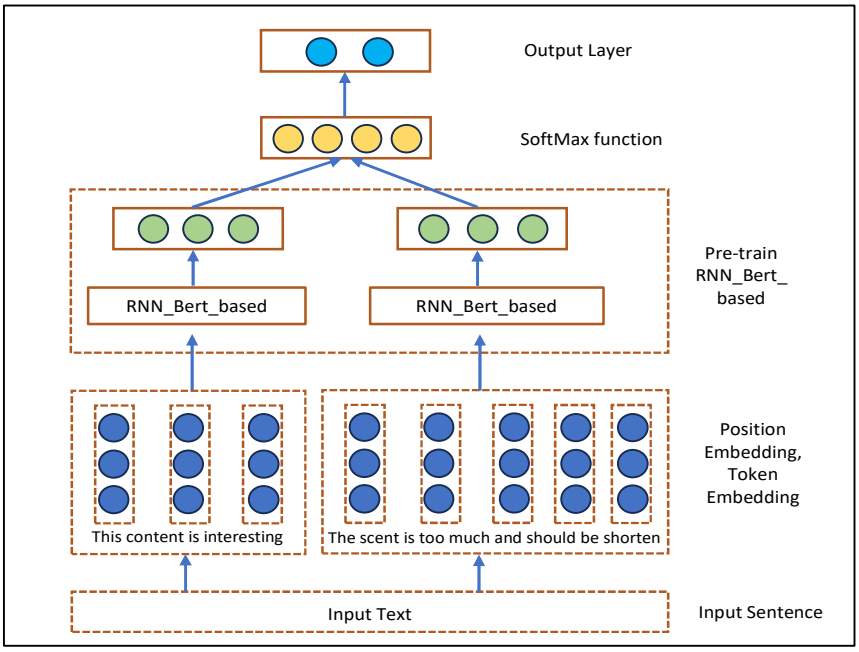


Figure 6. The integration of Bert and RNN.

3.5. Algorithm Flow of TextCNN + RoBERTa and RNN_Bert_Based

In Algorithm 1, the hybrid model integrates TextCNN and RoBERTa to capitalize on both deep contextual embeddings and convolutional feature extraction [14]. The RoBERTa model is a transformer-based model that provides powerful contextual embeddings by capturing deep semantic information from text. A transformer-based model provides robust contextual embeddings through the capture of profound semantic information from textual data. It is an enhanced iteration of Bert that has been fine-tuned on a substantial corpus for enhanced language comprehension. TextCNN is a convolutional neural network designed for the processing of textual data. Convolutional filters are applied to extract local features from the text embeddings, which can capture important patterns that are useful for text classification tasks.

The data is divided into the following categories: The dataset designated as "X_train_text" comprises the following elements: The input training texts are represented by the variable y_train. The labels correspond to the aforementioned training data. X_test_text is the input test text that the model will subsequently predict following the completion of the training phase. The input texts from the X_train_text (training) and X_test_text (testing) datasets are provided as input to the RoBERTa model. The RoBERTa model processes each input text and generates a sequence of hidden states, or embeddings, for each token within the text. The most common practice is to take the output from the [CLS] token or apply some form of pooling (such as mean pooling) to obtain a fixed-size vector representing the entire sentence or document. The embeddings from RoBERTa are then passed through the TextCNN component. Subsequently, a max-over-time pooling operation is typically employed to retain the most salient features across the entire text sequence. The outputs from the various convolutional filters are then concatenated to form a single feature vector. This vector represents the most salient features of the text as identified by the convolutional filters. A learning rate (lr) of 0.00002 is employed, which is a standard value for fine-tuning transformer-based models such as RoBERTa. It determines the extent to which the model's weights are updated during the training phase. The model will undergo five iterations over the entirety of the training dataset, which is equivalent to the number of epochs. A batch size of 16 or 32 signifies the number of samples that the model processes prior to updating its weights. Typically, transformer-based models demonstrate enhanced performance when trained with larger batch sizes (e.g., 32). However, this depends on the availability of computational resources. The binary cross-entropy loss function is employed, which is a

standard approach for binary classification tasks. It quantifies the discrepancy between the predicted probabilities and the actual binary labels. The provided parameters (learning rate, epoch, batch size, etc.) are set with the objective of optimizing the training process, with the aim of achieving good generalization and performance on the test set.

Algorithm 1 TextCNN + RoBERTa

Data: X_train_text, y_train, X_test_text
Result: Trained Hybrid Model, Predictions
Initialize TextCNN model:
    TextCNN (The size of word embeddings, 2D convolutional layers, classification output)
    Initialize RoBERTa model:
    roberta_model = loads a pre-trained RoBERTa model
    hybrid_model = creates a hybrid model by combining a TextCNN mode
    Training parameters:
    Learning rate = 2×10^{-5}, Epochs = 5, and Batch_size = 16, 32
    Loss function and optimizer:
    criterion = Binary Cross-Entropy Loss ()
    optimizer = Adam
    Training loop:
    for epoch in range(epochs): **do**
        hybrid_model.train()
        For
            i in tqdm(range(0, len(X_train_text), batch_size)):
            Do
                Text input batch = X_train_text, (*i*: *i*+batch_size)
                Inputs for the RoBERTa batch = additional input features for RoBERTa
                labels_batch = True labels for the current batch of data.
                clears the old gradients from the previous iteration()
                Runs the current batch through the hybrid model, producing predictions.
                loss = computes the loss via Binary Cross-Entropy Loss ()
                backpropagation ()
                optimizer.step()
            end
        end
        Evaluate the hybrid model:
        hybrid model:
        Disables gradient calculation to save memory and computations ():
            Extracts the input_ids from the test data
            inputs other required parameters by the RoBERTa model (like attention masks or token type IDs)
            Converts the binary predictions to integer values 0 (negative) or 1 (positive)
        Metrics calculation

In Algorithm 2, the objective is to leverage the strengths of both the Bert model and the RNN by integrating them into a unified framework, namely RNN_Bert_based. This approach capitalizes on the deep contextual embeddings of the Bert model and the recurrent nature of the RNN, combining the two into a single, more powerful model. This configuration capitalizes on the respective strengths of both models. Bert is employed for the purpose of capturing rich contextual embeddings, while RNN is utilized for the handling of sequential dependencies over these embeddings. The Bert model is a transformer-based model that generates robust contextualized embeddings for text by processing the entire sequence of tokens simultaneously and capturing bidirectional context. RNN is a type of artificial neural network designed to process sequential data. It maintains a hidden state that captures information about previous inputs, making it ideal for modeling dependencies over time or sequences. The dataset is divided into the following categories: The dataset, designated as "X_train_text", is comprised of the input training texts and is represented by the variable y_train (the aforementioned training data is accompanied by

the corresponding labels); and the X_test_text category is comprised of the input test texts, which the model will subsequently predict following the completion of the training phase.

Bert tokenizes the input text, producing embeddings for each token in the sequence. The resulting embeddings are context-rich, as Bert considers the entire sequence of words when generating each token's embedding. The output from Bert can be either the hidden states of all tokens or, alternatively, the hidden state corresponding to the [CLS] token, which often serves as a summary of the entire sequence. The RNN layer has a function for passing the embeddings from Bert through RNN. This could be a basic RNN, such as a long short-term memory (LSTM) network. The basic recurrent network has the potential for vanishing gradients, which is a mechanism for retaining information over long sequences. This mitigates issues in the proposed model. The RNN processes the sequence of Bert embeddings, updating its hidden state with each token, and captures sequential dependencies that Bert might not explicitly model, given that Bert is bidirectional but not inherently sequential. The final hidden state of the RNN (or a combination of hidden states, such as the output at each time step) is then extracted. The aforementioned hidden state represents the entirety of the sequence as processed by the RNN, incorporating both the contextual information derived from Bert and the sequential dependencies captured by the RNN.

Algorithm 2 RNN_Bert_based

Data: X_train_text, y_train, X_test_text
Result: Trained RNN_Bert_based, Predictions
Initialize RNN_Bert_based **model:**
    Bert (Setting up parameters for Bert model)
    Initialize Bertmodel:
    Bert_model = loads a pre-trained Bert model
    Integrated_model = Integrated_Model l (Bert_model, RNN_model)
    Training parameters:
    Learning rate = 2×10^{-5}, Epochs = 5, and Batch_size = 16, 32
    Loss function and optimizer:
    criterion = Cross Entropy Loss ()
    optimizer = Adam
    Training loop:
    for epoch in range(epochs): **do**
        Model Training
        For (Trains the model for a specified number of epochs until validation accuracy improves.)
            i in tqdm (range(0, len(X_train_text), batch_size)):
            Do
                Data: An RNN model M with L layers, Weight W, pruning rate P, training epoch n
                For $i \leftarrow 0$ to L & epoch $\leftarrow 0$ *to* n to **do**
                    W $\leftarrow$ Initiallization Weight
                    W $\leftarrow$ normal training (W)
                    % Weight prunning
                    labels_batch = True labels for the current batch of data.
                    labels_batch = y_train[*i*: *i* + batch_size]
                    Optimizer = Adam
                    loss = Cross Entropy Loss ()
                    loss.backward()
                    optimizer.step()
                end
            end
        end
        Metrics calculation: outputs = binary predictions to integer values 0 (negative) or 1 (positive)

The hidden state of the RNN is passed through a fully connected dense layer. This layer serves to map the high-level features extracted by the RNN to the output space. The final output is a sigmoid-activated unit, given that the loss function is the Binary Cross-Entropy Loss. This unit produces a probability for the binary classification task. A learning rate (lr) of 0.00002 is employed, which is a standard value for fine-tuning transformer-based models such as RoBERTa. This parameter determines the extent to which the model's weights are updated during the training phase. The model will be trained on the entire training dataset, comprising five iterations or epochs. A batch size of 16 or 32 represents the number of samples that the model processes prior to updating its weights. It is typically observed that transformer-based models exhibit enhanced performance when trained with larger batch sizes, for example 32. However, this is contingent upon the availability of sufficient computational resources. The binary cross-entropy loss function is utilized, which represents a conventional methodology for binary classification tasks. It quantifies the discrepancy between the predicted probabilities and the actual binary labels. The provided parameters (learning rate, epoch, batch size, etc.) are set with the objective of optimizing the training process, with the aim of achieving good generalization and performance on the test set.

4. Experiment Result and Analysis

To ensure the effective implementation of the RNN_Bert_based algorithm, it is essential to employ the traditional Bert model in conjunction with the RNN. The objective of this integration is to enhance the model's capacity to identify and capture sequential dependencies within textual data. The SST-2 dataset which is widely recognized for its binary sentiment classification tasks will serve as the primary benchmark for evaluating the algorithm's performance. The objective is to enhance the model's accuracy and robustness in sentiment analysis by merging Bert's contextual embeddings with RNN's sequential learning capabilities. The following section provides a detailed explanation and analysis of the evaluation metrics, the proposed method, and a comprehensive overview of existing work on text classification models. The objective of this section is to provide a comprehensive understanding of the methodologies employed, offering insights into the strengths and limitations of the proposed approach in comparison with previously established models.

4.1. Datasets and Evaluation Metrics

In this paper, the SST-2 [39] dataset is selected as the experimental dataset. The SST-2 dataset is a challenging dataset for text classification tasks. The dataset is a popular resource for sentiment analysis, consisting of 11,855 movie review sentences labeled for binary sentiment classification as either positive or negative. The dataset is split into a training set of 8544 sentences, a testing set of 2210 sentences, and a validation set of 1101 sentences.

A large portion of the dataset with 72% is allocated to training. This ensures that the model is exposed to a diverse range of examples, allowing it to generalize well to new data. The more data it sees during training, the better it learns the underlying patterns in the data. Next, a smaller proportion, about 9% of the dataset, is allocated to validation. This is sufficient to monitor the model's performance without wasting too much data that could have been used for training. The validation set helps prevent overfitting by providing a reality check on the model's performance on unseen data. This way, the model does not just memorize the training data but generalizes well to new examples. It also aids in tuning the model's hyperparameters for optimal performance. In this case, the test set is around 18% of the total data. This amount is sufficient to give a reliable estimate of model performance across a variety of examples while still preserving enough data for training and validation. Figure 1 illustrates the labeling and sentence length analysis of the SST-2 training set.

In classification problems, metrics such as accuracy, precision, recall, and F1 score are typically employed to assess the performance of classification models. According to the confusion matrix, the experimental results can be classified into four categories: true

positive (TP), false positive (FP), true negative (TN), and false negative (FN). Among these, TP denotes the number of texts with positive actual labels and positive model predictions; FP denotes the number of texts with negative actual labels but positive model predictions; FN denotes the number of texts with positive actual labels but negative model predictions; TN denotes the number of texts with negative actual labels and negative model predictions. The accuracy, precision, recall, and F1 score are calculated as follows:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} * 100\% \quad (4)$$

$$Precision = \frac{TP}{TP + FP} * 100\% \quad (5)$$

$$Recal = \frac{TP}{TP + FN} * 100\% \quad (6)$$

$$F1\ score = \frac{2 * Precision * Recal}{Precision + Recal} * 100\% \quad (7)$$

The above formula indicates that the accuracy is the ratio of correctly classified samples to the total number of samples. This reflects the overall performance of the classifier. However, when there is a category imbalance in the dataset, the accuracy rate may be distorted because the classifier may tend to predict the category with a higher number of samples. Accuracy is the percentage of samples that the classifier predicts to be positive that are actually positive cases. The recall rate is the proportion of positive examples that are correctly identified as such by the classifier. When the samples are imbalanced, the precision and recall rates can provide a more accurate reflection of the classification performance of the model, although they may not fully capture the prediction ability of negative samples. The F1 score is the average of the precision and recall rates, which can combine the prediction accuracy and recognition ability of the classifier. However, the F1 score assigns equal weight to precision and recall, which may result in the overlooking of certain metrics that should be accorded greater importance. To summarize, each of the four metrics, such as accuracy, precision, recall, and F1 score, has its own set of advantages and disadvantages. In order to comprehensively evaluate the classification performance of the model, the aforementioned four metrics are adopted as the evaluation metrics for this experiment.

4.2. Experiments and Parameter Settings

In order to comprehensively evaluate the effectiveness of the proposed model in extracting both local and global high-level contextual semantic information and to compare its performance against traditional classification models, a series of experimental studies were undertaken. Additionally, this study includes a comparison with several statistically-based traditional machine learning classification models, including XLNet, TextCNN + RoBERTa, Bert, and ALBERT (A Lite Bert), in addition to benchmarking against the original Bert base.

1. Bert was introduced by Google in 2018 and has revolutionized natural language understanding tasks. It is based on the Transformer architecture and trained on two unsupervised tasks: Masked Language Model (MLM) and Next Sentence Prediction (NSP). Bert is capable of understanding context by considering both the left and right context of a word. Nevertheless, it is constrained by the fixed-length input limitation inherent to its token-based approach.
2. The TextCNN + RoBERTa (Convolutional Neural Networks for Efficient Text Classification and Robus Bidirectional Encoder Representations from Transformers) was proposed by [14], which can adapt convolutional neural networks (CNNs). It was originally designed for image data to handle text data. CNNs are effective at capturing local patterns, which makes them well-suited for tasks where n-grams (sequences of words) are of importance. RoBERTa is an enhanced version of Bert that was developed to refine the original Bert model through optimized pre-training. In 2019, Facebook

AI introduced RoBERTa, which modifies Bert in several ways to achieve enhanced performance on a range of NLP tasks. The combination of TextCNN and RoBERTa offers several benefits, including the following:

a. Complementary Strengths: While RoBERTa demonstrates proficiency in discerning contextual relationships within textual data, it may exhibit a deficiency in identifying specific local patterns, such as n-grams, that are pivotal for certain tasks. In contrast, TextCNN demonstrates a high proficiency in capturing local patterns, although it is less adept at grasping the intricate contextual nuances that RoBERTa is capable of discerning.
b. Enhanced Feature Representation: The combination of these models allows for the exploitation of RoBERTa's profound, context-aware representations, which are augmented by the capacity of TextCNN to discern salient local features. The combination of these models may result in enhanced classification performance, particularly in tasks where both global context and local patterns are of significance.
c. The TextCNN + RoBERTa hybrid model is an innovative approach that effectively integrates the strengths of convolutional neural networks and transformer-based architectures, resulting in a robust and efficient solution for text classification tasks. This combination enables the model to capture both local and global features in text data, which may result in enhanced performance across a diverse range of NLP applications.

3. XLNet, which was introduced by Google in 2019, represents an improvement upon Bert's bidirectional context understanding. It employs a permutation-based approach, leveraging an autoregressive model similar to the Transformer-XL architecture. This enables the capture of bidirectional context without the need for masked language modeling. The enhanced bidirectional context understanding results in enhanced performance on various downstream tasks.
4. ALBERT (A Lite BERT), which was proposed by Google in 2019, is designed to reduce the computational cost of Bert while maintaining or even improving its performance. This is achieved by factorizing the embedding parameters and applying cross-layer parameter sharing. ALBERT also introduces two novel training strategies: Sentence Order Prediction and Cross-Example Loss, which are intended to further enhance its performance. ALBERT reduces the number of parameters compared to Bert by sharing parameters across layers. In Bert, each layer has its own set of parameter weights, but ALBERT reuses the same parameters for each layer. This significantly reduces the total number of parameters, making the model smaller and faster. It splits the large vocabulary embedding matrix into two smaller matrices, which also helps in reducing the model size. This factorization decouples the size of the hidden layers from the size of the vocabulary embeddings, allowing for a smaller overall footprint.

4.3. Experimental Results

The experiment is conducted under a variety of conditions, including the Bert-based method with different learning rates and batch sizes, the KNN_Bert_based method, the proposed RNN_Bert_based method, and other existing text classification methods for comparison with the proposed model.

4.3.1. Bert-Based Method with Different Learning Rates and Batch Sizes

The following results present an evaluation of the Bert model's accuracy for text classification, with varying learning rates and total batch sizes of 16 for each experiment. The results demonstrate that with a total batch size of 16 and learning rates of 2×10^{-5}, 3×10^{-5}, and 5×10^{-5}, accuracies of 92.75%, 92.20%, and 92.31% can be achieved, respectively. Figure 7a presents a comparison of accuracy across different learning rates with a batch size of 16.

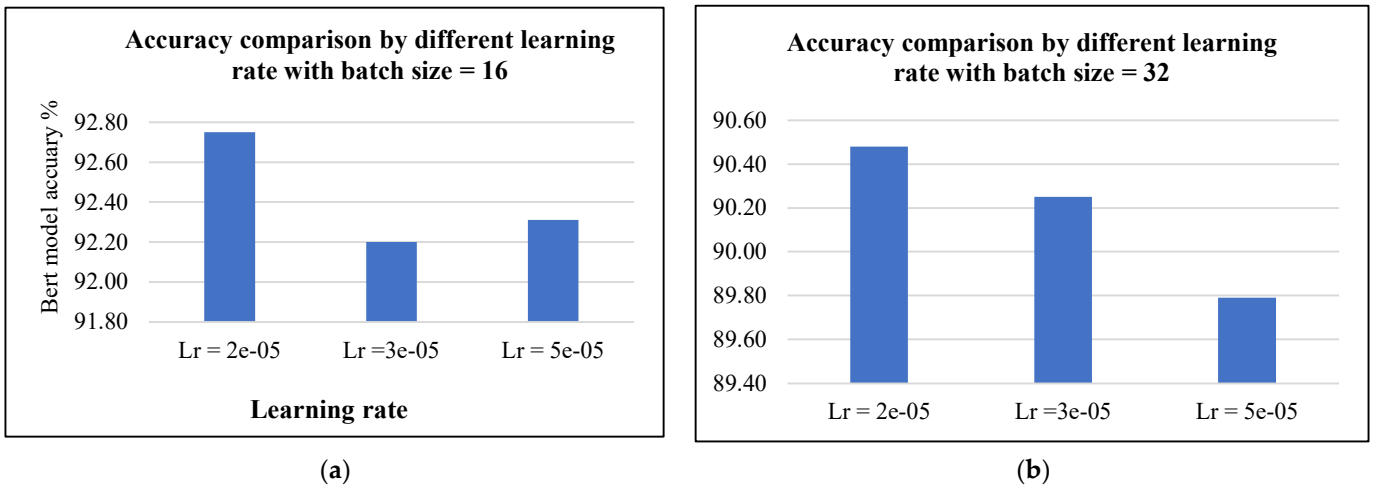


Figure 7. Bert model accuracy comparison by different learning rates with batch size = 16 (**a**), Bert model accuracy comparison by different learning rates with batch size = 32 (**b**).

The next results present an evaluation of the Bert model's accuracy for text classification, with varying learning rates and total batch sizes of 32 for each experiment. The results demonstrate that with a total batch size of 32 and learning rates of 2×10^{-5}, 3×10^{-5}, and 5×10^{-5}, accuracies of 90.48%, 90.25%, and 89.79% can be achieved, respectively. Figure 7b presents a comparison of accuracy across different learning rates with a batch size of 32.

The outcome of the experiment, conducted with a batch size of 32, indicates that an increase in the learning rate is associated with a decline in the performance of the model. The results indicate that the learning rate of 2×10^{-5} with a batch size of 16 represents the optimal Bert hyperparameters. This configuration will serve as the primary approach in the integration with RNN, with the objective of achieving enhanced performance in model accuracy for classification tasks.

4.3.2. KNN_Bert_Based Method

The integration of KNN with the Bert model leverages the strengths of both techniques, enhancing semantic understanding, improving performance, providing robustness, and maintaining simplicity and efficiency. This integration is particularly powerful for tasks that require high-quality text representations and flexible, interpretable classification or retrieval methods. The experiment involved training a Bert model and a K-Nearest Neighbors (KNN) model with the following key parameters for the Bert model part: max_seq_len = 50, epochs = 5, batch_size = 16, learning rate $(lr) = 2 \times 10^{-5}$, and patience = 1. For the KNN model part, the number of neighbors to use for the K-Nearest Neighbors algorithm is 5. This is a critical hyperparameter for KNN that determines how many nearest neighbors are considered when making a prediction. Here, n_neighbors = 5 means that the KNN model will look at the 5 nearest points in the training data when predicting the label for a new data point. The following results present an evaluation of the KNN_Bert_based model's accuracy for text classification, with a learning rate (lr) of 2×10^{-5} and a total batch size of 16 in five episodes. This is a hybrid model that combines KNN with Bert, a powerful pre-trained transformer model for natural language processing. This model is being evaluated for its performance in text classification, which means categorizing text data into predefined labels. The learning rate controls how much the model's weights are adjusted with respect to the gradient during training. Next is epoch which is the number of times the model goes through the entire dataset during training.

The results demonstrate that with a total batch size of 16 and learning rates of 2×10^{-5}, accuracies of 91.05, 92.26, 92.27, 92.30, and 92.31 were obtained from episodes 1 to 5, respectively. Figure 8 presents a comparison of the KNN_Bert_based model's accuracy with five epochs.

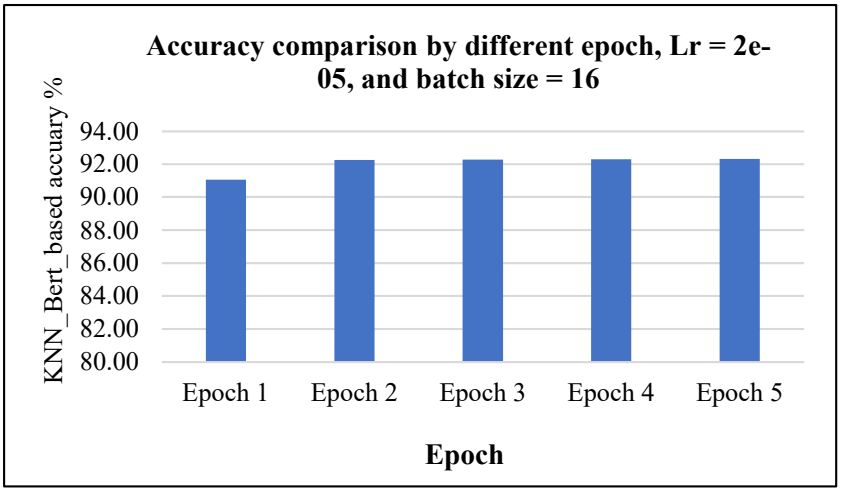


Figure 8. KNN_Bert_based accuracy comparison by different epochs.

4.3.3. Proposed RNN_Bert_Based Method

The integration of KNN with the Bert model leverages the strengths of both techniques. The experiment involved training a Bert model and a Recurrent Neural Network based (RNN_Bert_based) model with the following key parameters for the Bert model part: max_seq_len = 50, epochs = 5, batch_size = 16, learning rate $(lr) = 2 \times 10^{-5}$, and patience = 1. For our RNN model, we set the number of hidden units (rnn_hidden_size) to 256, allowing the network to capture complex patterns in the data with a balanced computational cost. The parameter defining the number of classes for the final classification task (num_labels) was set to 2, indicating a binary classification problem. We specified the use of an LSTM (Long Short-Term Memory) layer as the type of RNN. LSTMs are particularly effective for managing long-term dependencies in sequential data due to their gating mechanisms, which regulate information flow and help mitigate issues like vanishing gradients. For more details of parameter values for the experiment, see Table 1.

This configuration ensured our model was well-suited for tasks that require understanding temporal patterns and making binary decisions. Figure 9 presents a comparison of RNN_Bert_based model's accuracy within five epochs.

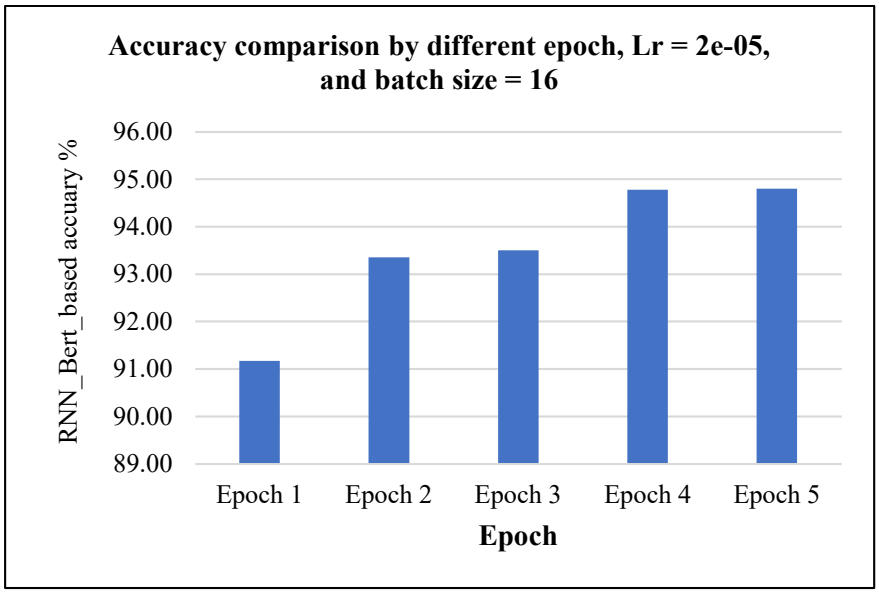


Figure 9. RNN_Bert_based accuracy comparison by different epochs.

Table 1. Parameter settings value for the model.

Number	Parameter	Value
1	Max_length	256
2	Batch_size	16
3	Learning_rate	2×10^{-5}
4	Dropout	0.5
5	Epoch	5
6	Activation Function	ReLU
7	Patience	1
8	max_seq_len	50

4.3.4. Comparison of Proposed RNN_Bert_Based and Other Methods

In this study, we conducted a comparative analysis with several established traditional machine learning classification models, including XLNet, TextCNN + RoBERTa, Bert, and ALBERT (A Lite BERT), comparing to KNN_Bert_based model and our proposed RNN_Bert_based model. The value of parameters utilized in experimentation was set according to the best Bert hyperparameters in our simulation to obtain model higher accuracy. These models represent a spectrum of approaches within the field, ranging from sophisticated transformer architectures like XLNet and ALBERT to more streamlined versions like TextCNN + RoBERTa. The primary objective of this comparison was to assess the performance of the proposed RNN_Bert_based model relative to well-established models across a range of experimental parameters. This comprehensive evaluation allowed us to explore the efficiency of our model in extracting both local and global high-level contextual semantic information in comparison to existing state-of-the-art methods.

Following the completion of the experiments and the subsequent analysis of the results, it was observed that the proposed model exhibited competitive performance across multiple evaluation metrics. In particular, it demonstrated superior performance in terms of accuracy, precision, recall, and F1 score in comparison to traditional methods. Furthermore, when benchmarked against the original Bert-based model, the proposed model demonstrated comparable or even improved performance, indicating its effectiveness in capturing nuanced semantic information and achieving robust classification results.

The experimental results of the proposed algorithm model and the comparison model on the SST-2 dataset, which is a benchmark dataset for sentiment analysis tasks, were comprehensively depicted and analyzed in Figure 10. The figure illustrates performance metrics of accuracy, highlighting the effectiveness and efficiency of the proposed model compared to the baseline or comparison model. The visual representation in Figure 10 also provides insight into the robustness of the models across different sentiment categories, further emphasizing the improvements or limitations observed during the experiments. The model accuracy of our proposed method, RNN_Bert_based, along with other leading models such as XLNet, TextCNN + RoBERTa, ALBERT, and KNN_Bert_based has been rigorously experimented on, and its results have been meticulously captured, analyzed, and documented for a comprehensive and in-depth comparison.

The experimental results demonstrated the effectiveness and potential of the proposed RNN_Bert_based model in addressing the complexity of sentiment analysis and classification tasks. This points out the way for advancements in our proposed model in NLP and machine learning research. Based on the results obtained from the evaluation metrics for Bert, XLNet, TextCNN + RoBERTa, ALBERT, KNN_Bert_based, and the proposed RNN_Bert_based, it is evident that the proposed RNN_Bert_based achieved the highest performance accuracy across all metrics. RNN_Bert_based can achieve an accuracy of 94.8%, KNN_Bert_based can achieve an accuracy of 92.31%, TextCNN + RoBERTa can achieve an accuracy of 94.00%, ALBERT can achieve an accuracy of 91.92%, Bert can achieve an accuracy of 92.15%, and XLNet showed a slightly lower accuracy of 84.16%, among other methods.

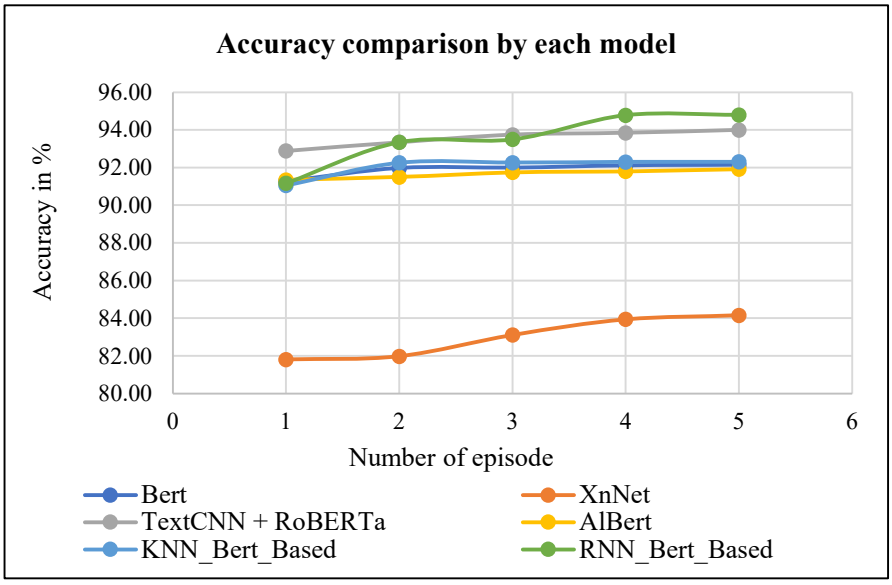


Figure 10. RNN_Bert_based accuracy comparison with other methods.

The subsequent experimental results present a detailed comparison of precision, recall, and F1 score metrics for each model when evaluated against the proposed RNN_Bert_based method. This comprehensive analysis demonstrates that the RNN_Bert_based model not only outperforms traditional models in terms of these key performance indicators but also highlights its potential to significantly enhance the field of NLP and machine learning research. The results demonstrate that the proposed model effectively captures and leverages the intricate patterns within the data, resulting in substantial improvements in model accuracy and generalization. Consequently, the model offers a robust solution for various NLP tasks. This development highlights the significance of integrating RNN architectures with Bert-based embeddings to advance current methodologies in this work. This improvement is evident in its ability to achieve the highest performance across all metrics, boasting the RNN_Bert_based model's precision, recall, and F1 score to reach the value of 94.80%. Additionally, comparing with other models, TextCNN + RoBERTa follows closely with a precision of 94.10%, recall of 94.0%, and F1 score of 94.0%. KNN_Bert_based also performs well with a precision of 92.10%, recall of 92.40%, and F1 score of 92.10%. ALBERT and Bert exhibit slightly lower but still notable performance, with ALBERT achieving a precision of 91.70%, recall of 91.80%, and F1 score of 91.70%, and Bert attaining a precision of 92.00%, recall of 92.30%, and F1 score of 92.00%. However, XLNet trails behind the others with a precision of 83.60%, recall of 83.60%, and F1 score of 83.95%. Figure 11 shows a comparison of the RNN_Bert_based model's precision with other methods.

The results clearly demonstrate that the RNN_Bert_based model exhibits a substantial degree of superiority over the other methods that were subjected to evaluation in this study. Figure 12a provides a detailed comparison of the recall achieved by the RNN_Bert_based model against other methods, demonstrating its superior performance. In particular, the RNN_Bert_based model exhibits a recall rate of 94.80%, which is markedly higher than that of the alternative models.

Similarly, Figure 12b presents a comparison of the F1 score obtained by the RNN_Bert_based model with those of other methods. The results clearly demonstrate that the RNN_Bert_based model not only matches but also exceeds the performance of the other models, achieving an impressive F1 score of 94.80%. This consistent performance across multiple metrics serves to underscore the effectiveness of the RNN_Bert_based model in the context of text classification tasks, thereby rendering it a more robust and reliable approach in comparison to existing methods.

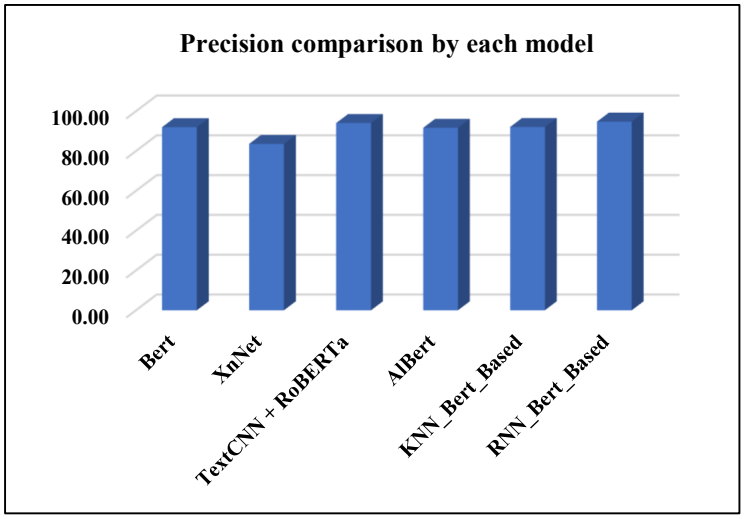
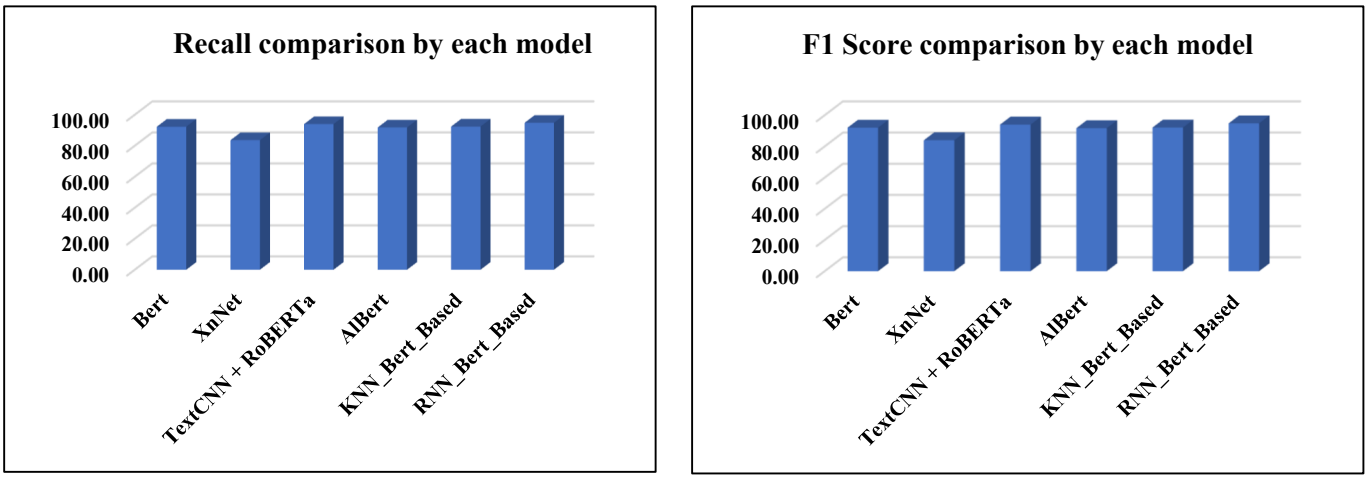


Figure 11. A comparison of the RNN_Bert_based model's precision with other methods.



(**a**) (**b**)

Figure 12. A comparison of RNN_Bert_based model's recall with other methods (**a**), A comparison of RNN_Bert_based model's F1 score with other methods (**b**).

5. Conclusions

In conclusion, this paper presents a text classification model based on the RNN_Bert_based architecture. The results of experimentation clearly demonstrate the superiority of the proposed RNN_Bert_based model in sentiment analysis and classification tasks. With an accuracy of 94.8%, it outperforms other models such as KNN_Bert_based, TextCNN + RoBERTa, ALBERT, and XLNet in key metrics including precision, recall, and F1 score. This indicates the effectiveness of integrating RNN architectures with Bert-based embeddings, offering substantial improvements in accuracy and generalization. The findings underscore the model's potential to significantly enhance NLP and machine learning research, providing a robust solution for various text classification challenges. The performance evaluation underscores the advancements made in NLP through the development of the proposed RNN_Bert_based model to overcome the existing work of TextCNN + RoBERTa [14], ALBERT, XLNet, Bert, and a newly developed model of KNN_Bert_based model. Each of the two models can provide significant improvements over traditional approaches and demonstrate their effectiveness in various NLP tasks. In the future, we also aim to explore enhancing Bert by integrating advanced AI techniques, such as reinforcement learning or attention-based mechanisms, for more efficient text classification, so that we can obtain an approach that could lead to more accurate, faster, and scalable text classification,

particularly in specialized domains like medical diagnostics, legal document analysis, or sentiment detection in social media.

Author Contributions: Data collection, C.E.; conceptualization, C.E.; methodology, C.E.; experimentation, C.E.; writing—original draft preparation, C.E.; software development and analysis, C.E.; providing research direction and checking on papers, S.L.; review, S.L.; editing, S.L.; investigation, S.L.; validation, S.L.; project administration, S.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Munikar, M.; Shakya, S.; Shrestha, A. Fine-grained Sentiment Classification using BERT. *arXiv* **2019**, arXiv:1910.03474.
2. Shiva Prasad, K.M.; Reddy, H. Text Mining: Classification of Text Documents Using Granular Hybrid Classification Technique. *Int. J. Res. Advent Technol.* **2019**, *7*, 1–8. [CrossRef]
3. Semary, N.A.; Ahmed, W.; Amin, K.; Pławiak, P.; Hammad, M. Enhancing Machine Learning-Based Sentiment Analysis through Feature Extraction Techniques. *PLoS ONE* **2024**, *19*, e0294968. [CrossRef] [PubMed]
4. Manasa, R. Framework for Thought to Text Classification. *Int. J. Psychosoc. Rehabil.* **2020**, *24*, 418–424. [CrossRef]
5. Challenges in text classification using machine learning techniques. *Int. J. Recent Trends Eng. Res.* **2018**, *4*, 81–83.
6. News Text Classification Model Based on Topic Model. *Int. J. Recent Trends Eng. Res.* **2017**, *3*, 48–52.
7. Lee, S.A.; Shin, H.S. Combining Sentiment-Combined Model with Pre-Trained BERT Models for Sentiment Analysis. *J. KIISE* **2021**, *48*, 815–824. [CrossRef]
8. Zhang, T.; Kishore, V.; Wu, F.; Weinberger, K.Q.; Artzi, Y. BERTScore: Evaluating Text Generation with BERT. *arXiv* **2020**, arXiv:1904.09675.
9. Sellam, T.; Das, D.; Parikh, A. BLEURT: Learning Robust Metrics for Text Generation. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 9 April 2020. [CrossRef]
10. Liu, S.; Tao, H.; Feng, S. Text Classification Research Based on Bert Model and Bayesian network. In Proceedings of the 2019 Chinese Automation Congress (CAC), Hangzhou, China, 22–24 November 2019; pp. 5842–5846. [CrossRef]
11. Hao, M.; Wang, W.; Zhou, F. Joint Representations of Texts and Labels with Compositional Loss for Short Text Classification. *J. Web Eng.* **2021**, *20*, 669–688. [CrossRef]
12. Bai, J.; Li, X. Chinese Multilabel Short Text Classification Method Based on GAN and Pinyin Embedding. *IEEE Access* **2024**, *12*, 83323–83329. [CrossRef]
13. Jiang, D.; He, J. Tree Framework with BERT Word Embedding for the Recognition of Chinese Implicit Discourse Relations. *IEEE Access* **2020**, *8*, 162004–162011. [CrossRef]
14. Salim, S.; Lahcen, O. A BERT-Enhanced Exploration of Web and Mobile Request Safety through Advanced NLP Models and Hybrid Architectures. *IEEE Access* **2024**, *12*, 76180–76193. [CrossRef]
15. Yu, S.; Su, J.; Luo, D. Improving BERT-Based Text Classification with Auxiliary Sentence and Domain Knowledge. *IEEE Access* **2019**, *7*, 176600–176612. [CrossRef]
16. Zhang, H.; Shan, Y.; Jiang, P.; Cai, X. A Text Classification Method Based on BERT-Att-TextCNN Model. In Proceedings of the 2022 IEEE 5th Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC), Chongqing, China, 16–18 December 2022; pp. 1731–1735. [CrossRef]
17. Alagha, I. Leveraging Knowledge-Based Features with Multilevel Attention Mechanisms for Short Arabic Text Classification. *IEEE Access* **2022**, *10*, 51908–51921. [CrossRef]
18. She, X.; Chen, J.; Chen, G. Joint Learning with BERT-GCN and Multi-Attention for Event Text Classification and Event Assignment. *IEEE Access* **2022**, *10*, 27031–27040. [CrossRef]
19. Meng, Q.; Song, Y.; Mu, J.; Lv, Y.; Yang, J.; Xu, L.; Zhao, J.; Ma, J.; Yao, W.; Wang, R.; et al. Electric Power Audit Text Classification with Multi-Grained Pre-Trained Language Model. *IEEE Access* **2023**, *11*, 13510–13518. [CrossRef]
20. Talaat, A.S. Sentiment Analysis Classification System Using Hybrid BERT Models. *J. Big Data* **2023**, *10*, 110. [CrossRef]
21. Garrido-Muñoz, I.; Montejo-Ráez, A.; Martínez-Santiago, F.; Ureña-López, L.A. A Survey on Bias in Deep NLP. *Appl. Sci.* **2021**, *11*, 3184. [CrossRef]
22. Wu, Y.; Jin, Z.; Shi, C.; Liang, P.; Zhan, T. Research on the Application of Deep Learning-based BERT Model in Sentiment Analysis. *arXiv* **2024**, arXiv:2403.08217.

23. Li, Q.; Li, X.; Du, Y.; Fan, Y.; Chen, X. A New Sentiment-Enhanced Word Embedding Method for Sentiment Analysis. *Appl. Sci.* **2022**, *12*, 10236. [CrossRef]
24. Chen, X.; Cong, P.; Lv, S. A Long-Text Classification Method of Chinese News Based on BERT and CNN. *IEEE Access* **2022**, *10*, 34046–34057. [CrossRef]
25. Chen, X.; Zhang, W.; Pan, S.; Chen, J. Solving Data Imbalance in Text Classification with Constructing Contrastive Samples. *IEEE Access* **2023**, *11*, 90554–90562. [CrossRef]
26. He, A.; Abisado, M. Text Sentiment Analysis of Douban Film Short Comments Based on BERT-CNN-BiLSTM-Att Model. *IEEE Access* **2024**, *12*, 45229–45237. [CrossRef]
27. Tang, T.; Tang, X.; Yuan, T. Fine-Tuning BERT for Multi-Label Sentiment Analysis in Unbalanced Code-Switching Text. *IEEE Access* **2020**, *8*, 193248–193256. [CrossRef]
28. Zhang, H.; Sun, S.; Hu, Y.; Liu, J.; Guo, Y. Sentiment Classification for Chinese Text Based on Interactive Multitask Learning. *IEEE Access* **2020**, *8*, 129626–129635. [CrossRef]
29. Peng, C.; Wang, H.; Wang, J.; Shou, L.; Chen, K.; Chen, G.; Yao, C. Learning Label-Adaptive Representation for Large-Scale Multi-Label Text Classification. *IEEE/ACM Trans. Audio Speech Lang. Process.* **2024**, *32*, 2630–2640. [CrossRef]
30. Lee, D.H.; Jang, B. Enhancing Machine-Generated Text Detection: Adversarial Fine-Tuning of Pre-Trained Language Models. *IEEE Access* **2024**, *12*, 65333–65340. [CrossRef]
31. Wang, Z.; Zheng, X.; Zhang, J.; Zhang, M. Three-Branch BERT-Based Text Classification Network for Gastroscopy Diagnosis Text. *Int. J. Crowd Sci.* **2024**, *8*, 56–63. [CrossRef]
32. Rehan, M.; Malik, M.S.I.; Jamjoom, M.M. Fine-Tuning Transformer Models Using Transfer Learning for Multilingual Threatening Text Identification. *IEEE Access* **2023**, *11*, 106503–106515. [CrossRef]
33. Kowsher, M.; Sami, A.A.; Prottasha, N.J.; Arefin, M.S.; Dhar, P.K.; Koshiba, T. Bangla-BERT: Transformer-Based Efficient Model for Transfer Learning and Language Understanding. *IEEE Access* **2022**, *10*, 91855–91870. [CrossRef]
34. Xiao, H.; Luo, L. An Automatic Sentiment Analysis Method for Short Texts Based on Transformer-BERT Hybrid Model. *IEEE Access* **2024**, *12*, 93305–93317. [CrossRef]
35. Olivato, M.; Putelli, L.; Arici, N.; Gerevini, A.E.; Lavelli, A.; Serina, I. Language Models for Hierarchical Classification of Radiology Reports with Attention Mechanisms, BERT, and GPT-4. *IEEE Access* **2024**, *12*, 69710–69727. [CrossRef]
36. Corizzo, R.; Leal-Arenas, S. One-Class Learning for AI-Generated Essay Detection. *Appl. Sci.* **2023**, *13*, 7901. [CrossRef]
37. Perera, P.; Patel, V.M. Learning Deep Features for One-Class Classification. *IEEE Trans. Image Process.* **2019**, *28*, 5450–5463. [CrossRef]
38. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778.
39. Socher, R.; Bauer, J.; Manning, C.D.; Ng, A.Y. Parsing with compositional vector grammars. In Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics, Sofia, Bulgaria, 4–9 August 2013; Volume 1, pp. 455–465, Long Papers.